

SARIMA_SEMANAL_ALIM

May 2, 2026

1 Estacionalidad con indice promedio de ingresos

```
[3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import warnings
warnings.filterwarnings('ignore')

from statsmodels.tsa.seasonal import seasonal_decompose, STL

# 1. CARGAR DATOS
df = pd.read_csv('Ingresos Alimentadoras.csv', encoding='latin1')

for col in ['Monto Total Efectivo', 'Monto Total Tarjetas']:
    df[col] = df[col].replace(r'[\$,]', '', regex=True)
    df[col] = df[col].replace(r'^\s*-\s*$', '0', regex=True)
    df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)

df['Ingreso Total'] = df['Monto Total Efectivo'] + df['Monto Total Tarjetas']
df['Fecha'] = pd.to_datetime(df['Fecha'], dayfirst=True)

# Serie diaria (necesaria para extraer día de la semana)
ts_diaria = df.groupby('Fecha')['Ingreso Total'].sum()
ts_diaria = ts_diaria.asfreq('D').interpolate()

# Serie semanal
ts_semanal = ts_diaria.resample('W').sum()
ts_semanal = ts_semanal[(ts_semanal.index >= '2023-01-01') & (ts_semanal.index_
    <= '2025-12-31')]

print(f"Rango: {ts_semanal.index.min().date()} → {ts_semanal.index.max().
    <date()}")
print(f"Total semanas: {len(ts_semanal)}\n")

# A) ESTACIONALIDAD POR DÍA DE LA SEMANA
```

```

dias_nombres = ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado',
↳ 'Domingo']

ts_dia = ts_diaria[(ts_diaria.index >= '2023-01-01') & (ts_diaria.index <=
↳ '2025-12-31')].copy()
ts_dia = ts_dia.to_frame(name='Ingreso')
ts_dia['DiaSemana'] = ts_dia.index.dayofweek # 0=Lun ... 6=Dom
ts_dia['DiaNombre'] = ts_dia['DiaSemana'].map(dict(enumerate(dias_nombres)))

# Promedio e índice estacional por día
promedio_dia = ts_dia.groupby('DiaSemana')['Ingreso'].mean()
media_global = ts_dia['Ingreso'].mean()
indice_dia = (promedio_dia / media_global * 100).rename('Índice')

print(" Índice estacional por día de la semana ")
print(" (100 = promedio global; >100 = día por encima del promedio)")
for d, idx in zip(dias_nombres, indice_dia):
    barra = ' ' * int(idx / 5)
    print(f" {d:<10} {idx:6.1f} {barra}")

# Gráfica A
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

colores = ['#4CAF50' if i == indice_dia.idxmax() else
↳ '#F44336' if i == indice_dia.idxmin() else
↳ '#2196F3' for i in indice_dia.index]

axes[0].bar(dias_nombres, indice_dia.values, color=colores, edgecolor='white',
↳ lw=0.8)
axes[0].axhline(100, color='black', linestyle='--', lw=1.2, label='Promedio =
↳ 100')
axes[0].set_title('Índice estacional por día de la semana', fontweight='bold')
axes[0].set_ylabel('Índice (100 = promedio)')
axes[0].legend()
axes[0].grid(axis='y', alpha=0.3)
for i, v in enumerate(indice_dia.values):
    axes[0].text(i, v + 1, f'{v:.1f}', ha='center', va='bottom', fontsize=9)

# Boxplot por día
data_box = [ts_dia[ts_dia['DiaSemana'] == d]['Ingreso'].values / 1e6 for d in
↳ range(7)]
bp = axes[1].boxplot(data_box, labels=dias_nombres, patch_artist=True,
↳ medianprops=dict(color='white', lw=2))
for patch, color in zip(bp['boxes'], colores):
    patch.set_facecolor(color)
    patch.set_alpha(0.8)

```

```

axes[1].set_title('Distribución de ingresos por día (millones MXN)',
    ↪fontweight='bold')
axes[1].set_ylabel('Millones MXN')
axes[1].grid(axis='y', alpha=0.3)

plt.suptitle('Estacionalidad Semanal - Alimentadoras 2023-2025', fontsize=13,
    ↪fontweight='bold')
plt.tight_layout()
plt.savefig('estacionalidad_dia_semana.png', dpi=150, bbox_inches='tight')
plt.close()
print("\n Guardado: estacionalidad_dia_semana.png")

#
# B) DESCOMPOSICIÓN STL DE LA SERIE SEMANAL
#     (tendencia + componente estacional + residuo)
#
# period=52 → estacionalidad anual expresada en semanas
stl = STL(ts_semanal, period=52, robust=True)
res_stl = stl.fit()

fig, axes = plt.subplots(4, 1, figsize=(14, 12), sharex=True)

axes[0].plot(ts_semanal.index, ts_semanal / 1e6, color='steelblue', lw=1.5)
axes[0].set_ylabel('MXN (M)')
axes[0].set_title('Serie original', fontweight='bold')
axes[0].grid(True, alpha=0.3)

axes[1].plot(res_stl.trend.index, res_stl.trend / 1e6, color='tomato', lw=2)
axes[1].set_ylabel('MXN (M)')
axes[1].set_title('Tendencia', fontweight='bold')
axes[1].grid(True, alpha=0.3)

axes[2].bar(res_stl.seasonal.index, res_stl.seasonal / 1e6,
    color='mediumseagreen', alpha=0.8, width=5)
axes[2].axhline(0, color='black', lw=0.8)
axes[2].set_ylabel('MXN (M)')
axes[2].set_title('Componente estacional (período 52 semanas)',
    ↪fontweight='bold')
axes[2].grid(True, alpha=0.3)

axes[3].plot(res_stl.resid.index, res_stl.resid / 1e6,
    color='gray', lw=1, alpha=0.8)
axes[3].axhline(0, color='black', lw=0.8, linestyle='--')
axes[3].set_ylabel('MXN (M)')
axes[3].set_title('Residuo', fontweight='bold')
axes[3].grid(True, alpha=0.3)

```

```

plt.suptitle('Descomposición STL - Serie semanal 2023-2025', fontsize=13,
    ↪fontweight='bold')
plt.tight_layout()
plt.savefig('descomposicion_stl.png', dpi=150, bbox_inches='tight')
plt.close()
print(" Guardado: descomposicion_stl.png")

# C) PATRÓN ESTACIONAL PROMEDIO POR SEMANA DEL AÑO (semanas 1-52)
#
ts_sem = ts_semanal.copy().to_frame(name='Ingreso')
ts_sem['SemanaAño'] = ts_sem.index.isocalendar().week.astype(int)
patron_semana = ts_sem.groupby('SemanaAño')['Ingreso'].mean()
indice_semana = patron_semana / patron_semana.mean() * 100

print("\n Índice estacional promedio por semana del año ")
print(" Semanas que más se desvían del promedio:")
top5_altas = indice_semana.nlargest(5)
top5_bajas = indice_semana.nsmallest(5)
print("\n TOP 5 semanas MÁS altas:")
for s, v in top5_altas.items():
    print(f" Semana {s:>2} → {v:.1f}")
print("\n TOP 5 semanas MÁS bajas:")
for s, v in top5_bajas.items():
    print(f" Semana {s:>2} → {v:.1f}")

fig, ax = plt.subplots(figsize=(16, 5))
colores_sem = ['#F44336' if v == indice_semana.max() else
    '#4CAF50' if v == indice_semana.min() else
    '#2196F3' for v in indice_semana.values]
ax.bar(indice_semana.index, indice_semana.values, color=colores_sem, alpha=0.
    ↪85, width=0.8)
ax.axhline(100, color='black', linestyle='--', lw=1.2, label='Promedio = 100')
ax.set_xlabel('Semana del año (ISO)')
ax.set_ylabel('Índice estacional')
ax.set_title('Patrón estacional por semana del año (promedio 2023-2025)',
    ↪fontweight='bold')
ax.set_xticks(range(1, 53))
ax.grid(axis='y', alpha=0.3)
ax.legend()
plt.tight_layout()
plt.savefig('patron_semanal_año.png', dpi=150, bbox_inches='tight')
plt.close()
print("\n Guardado: patron_semanal_año.png")

# D) EXPORTAR ÍNDICES A CSV

```

```

df_export = pd.DataFrame({
    'Semana_ISO': indice_semana.index,
    'Ingreso_Promedio': patron_semana.values,
    'Indice_Estacional': indice_semana.values
})
df_export.to_csv('indice_estacional_semanal.csv', index=False)
print(" Guardado: indice_estacional_semanal.csv")

print("\n" + "="*55)
print(" RESUMEN FINAL ".center(55, "="))
print("="*55)
print(f" Día más fuerte: {dias_nombres[indice_dia.idxmax()]} ({indice_dia.
    ↪max():.1f})")
print(f" Día más débil: {dias_nombres[indice_dia.idxmin()]} ({indice_dia.
    ↪min():.1f})")
print(f" Semana más alta: Semana {indice_semana.idxmax()} ({indice_semana.
    ↪max():.1f})")
print(f" Semana más baja: Semana {indice_semana.idxmin()} ({indice_semana.
    ↪min():.1f})")
print("="*55)

```

Rango: 2023-01-01 → 2025-12-28

Total semanas: 157

Índice estacional por día de la semana

(100 = promedio global; >100 = día por encima del promedio)

Lunes	107.4
Martes	109.1
Miércoles	108.2
Jueves	108.0
Viernes	110.1
Sábado	90.9
Domingo	66.2

Guardado: estacionalidad_dia_semana.png

Guardado: descomposicion_stl.png

Índice estacional promedio por semana del año

Semanas que más se desvían del promedio:

TOP 5 semanas MÁS altas:

Semana 48	→	110.2
Semana 36	→	109.0
Semana 46	→	108.3
Semana 35	→	108.0
Semana 45	→	107.9

TOP 5 semanas MÁS bajas:

Semana 52 → 77.4
Semana 1 → 82.0
Semana 14 → 87.7
Semana 16 → 91.6
Semana 29 → 93.3

Guardado: patron_semanal_año.png

Guardado: indice_estacional_semanal.csv

```
=====
===== RESUMEN FINAL =====
=====
Día más fuerte:    Viernes (110.1)
Día más débil:    Domingo (66.2)
Semana más alta:   Semana 48 (110.2)
Semana más baja:   Semana 52 (77.4)
=====
```

2 SARIMA con Pasajeros

```
[9]: import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
import warnings
warnings.filterwarnings('ignore')

# 1. Carga y preparación
df = pd.read_csv('Ingresos AlimentadoraS.csv', encoding='latin1')
df['Total Pasajes'] = df['Total Pasajes'].str.replace(',', '').astype(float)
df['Fecha'] = pd.to_datetime(df['Fecha'], dayfirst=True)

daily = df.groupby('Fecha')['Total Pasajes'].sum()
daily = daily.asfreq('D').fillna(0)

# 2. Entrenamiento 2023-2025
train = daily['2023':'2025']

# 3. Modelo SARIMA(0,1,1)(0,1,1)[7]
model = SARIMAX(train,
                 order=(0, 1, 1),
                 seasonal_order=(0, 1, 1, 7),
                 enforce_stationarity=False,
```

```

        enforce_invertibility=False)
result = model.fit(dis=False)
print(result.summary())

# 4. Promedio por día de semana (SIN excluir festivos)
DOW_ES = {0: 'Lun', 1: 'Mar', 2: 'Mié', 3: 'Jue', 4: 'Vie', 5: 'Sáb', 6: 'Dom'}
DOW_ORD = ['Lun', 'Mar', 'Mié', 'Jue', 'Vie', 'Sáb', 'Dom']

t = train.copy().to_frame()
t['dow'] = t.index.dayofweek

dow_avg = (t.groupby('dow')['Total Pasajes']
           .mean()
           .rename(index=DOW_ES)
           .reindex(DOW_ORD))

best_day = dow_avg.idxmax()
print("\n Promedio pasajes por día de semana (todos los días) ")
for day, val in dow_avg.items():
    flag = " MÁXIMO" if day == best_day else ""
    print(f" {day}: {val:,.0f}{flag}")

# 5. Gráfica de barras
BLUE = '#1C4E80'
GREEN = '#21A659'
BG = '#F4F7FB'

fig, ax = plt.subplots(figsize=(9, 6), facecolor=BG)
t = train.copy().to_frame()
t['dow'] = t.index.dayofweek

dow_avg = (t.groupby('dow')['Total Pasajes']
           .mean()
           .rename(index=DOW_ES)
           .reindex(DOW_ORD))

best_day = dow_avg.idxmax()

colors = [GREEN if d == best_day else BLUE for d in dow_avg.index]

bars = ax.bar(dow_avg.index, dow_avg.values / 1000,
              color=colors, edgecolor='white', linewidth=0.8,
              width=0.62, zorder=3)

# Etiquetas sobre cada barra
for bar, val in zip(bars, dow_avg.values):
    ax.text(bar.get_x() + bar.get_width() / 2,

```

```

        bar.get_height() + 0.5,
        f'{val / 1000:.1f}k',
        ha='center', va='bottom',
        fontsize=9.5, fontweight='bold', color='#1a1a1a')

# Títulos y ejes
ax.set_title('Promedio de pasajes por día de semana\n(2023-2025)',
             fontsize=13, fontweight='bold', color=BLUE, pad=12)
ax.set_ylabel('Pasajes (miles)', fontsize=10, color='#333')
ax.set_ylim(0, dow_avg.max() / 1000 * 1.20)
ax.tick_params(axis='x', labelsiz=10)
ax.tick_params(axis='y', labelsiz=9)
ax.grid(axis='y', linestyle='--', alpha=0.4, zorder=0)
ax.spines[['top', 'right', 'left']].set_visible(False)
ax.yaxis.set_tick_params(left=False)

# Nota al pie
ax.text(0.5, -0.12,
        f' Día con más ingresos: {best_day}',
        transform=ax.transAxes, ha='center',
        fontsize=12, fontweight='bold', color=GREEN)

# Nota modelo
ax.text(0.99, 0.97,
        'Modelo: SARIMA(0,1,1)(0,1,1)[7]',
        transform=ax.transAxes, ha='right', va='top',
        fontsize=8, color='#777',
        bbox=dict(boxstyle='round,pad=0.3', fc='#F0F4F8', ec='#ccc', lw=0.8))

plt.tight_layout()
plt.savefig('grafica_dow_sarima.png', dpi=150,
           bbox_inches='tight', facecolor=BG)
print("\n Guardada como 'grafica_dow_sarima.png'")

```

SARIMAX Results

```

=====
=====
Dep. Variable:                Total Pasajes    No. Observations:
1096
Model:                SARIMAX(0, 1, 1)x(0, 1, 1, 7)    Log Likelihood
-10209.507
Date:                Sat, 21 Mar 2026    AIC
20425.013
Time:                14:49:02    BIC
20439.965
Sample:                01-01-2023    HQIC
20430.675
- 12-31-2025

```



```

Covariance Type: opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ma.L1          -0.7036        0.015    -48.045      0.000     -0.732     -0.675
ma.S.L7        -0.8531        0.010   -85.274      0.000     -0.873     -0.833
sigma2         9.449e+06    1.64e+05    57.527      0.000    9.13e+06    9.77e+06
=====
===
Ljung-Box (L1) (Q):          17.24   Jarque-Bera (JB):
6467.23
Prob(Q):          0.00   Prob(JB):
0.00
Heteroskedasticity (H):      1.12   Skew:
-2.34
Prob(H) (two-sided):      0.30   Kurtosis:
14.05
=====
===

```

Warnings:

```
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
```

Promedio pasajes por día de semana (todos los días)

Lun: 33,229

Mar: 33,992

Mié: 33,741

Jue: 33,641

Vie: 34,067 MÁXIMO

Sáb: 27,674

Dom: 19,772

Guardada como 'grafica_dow_sarima.png'